

CO1-AT1-Short Answer Analysis

- 1) Suppose a modern OS supports multiple users and Processes with shared resources like CPU and I/O devices
- classify diff types of OS suitable for this scenario and justify your choice
 - Draw the structure of an OS
 - write a sequence of system calls required to create and execute a new process in such environment
- 2) consider an OS managing multiple files and devices
- calls used for file and device management
 - category of system calls used for file and D. management
 - illustrate how OS handles file operations using a layered structure diagram
 - write a pseudo-code or C-based system call to open read and close a file
- 3) Assume a real time OS used in an embedded system controlling industrial machinery
- identify the type of OS and explain why it is suitable for real time application
 - Draw a diagram representing the interaction between hardware, kernel and user processes
 - List and explain system calls required for process synchronization in this system.

4) A distributed computing system consists of multiple interconnected computers sharing resource over a network.

- (a) Classify the type of OS used and compare it with OS (b) Draw the architecture of distributed OS
(c) Write the sequence of system calls involved in inter process communication (IPC)

5) Consider a scenario where an OS must manage process scheduling, memory allocation and device communication efficiently.

- (a) Explain the major functions of an OS in tasks
(b) Draw a block diagram of OS components and interactions
(c) Write system calls examples for memory allocation

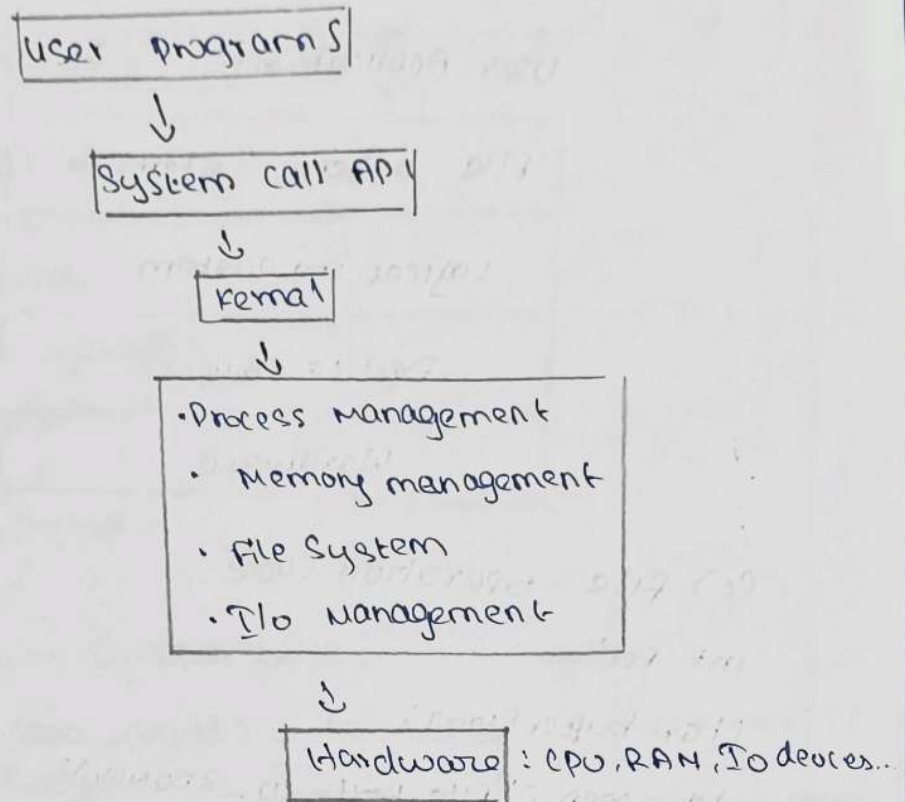
1) (a) Types of OS :

- Multiprogramming OS, Multi processing OS
- Time Sharing OS
- Multi user OS
- Distributed OS

Justification:

we need all 3. multi user for concurrent users. Time sharing for fair CPU allocation, Multiprocessing for performance when resources shared.

(b) OS Structure



(c) System call Sequence:

pid = getpid();

pid = fork();

if (pid == 0) {

exec("Program", "program", NULL);

}

else {

wait(NULL);

}

2) (a) File & Device system calls.

→ File management

• open()

• read()

• write()

• close()

• delete()

→ Device management

• ioctl()

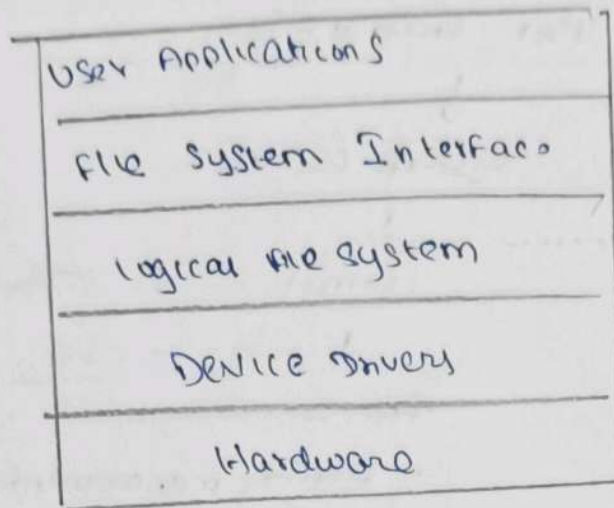
• read()

• write()

• open()

• close()

(b) Layered structure



(c) File operation code:

```
int fd;
```

```
char buffer[100];
```

```
fd = open ("file.txt", O_RDONLY);
```

```
read (fd, buffer, size of (buffer));
```

```
close (fd);
```

3) (a) Types of OS:

→ Real time operating system (RTOS)

Types: Hard RTOS (strict timing)

Soft RTOS (flexible timing)

Suitable because:

- Immediate response required

- Deterministic behaviour

- used in industrial control

(b) Interaction Diagram

User processes



kernel



• Scheduler

• Memory manager

• I/O control



Hardware

(c) Synchronization System calls :

• sem_init()



initialize

• sem_wait()



wait(lock)

• sem_post()



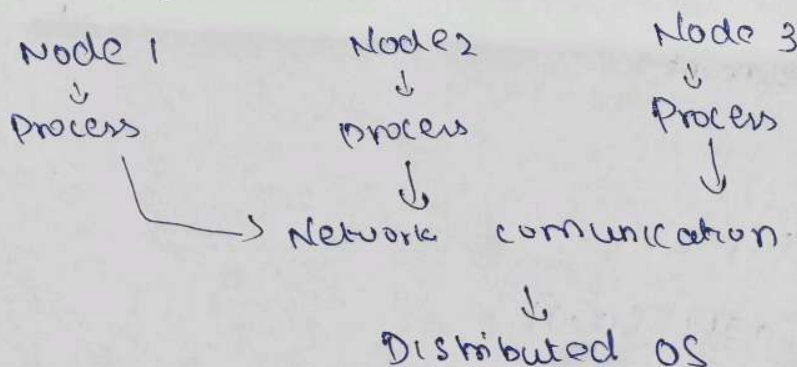
Signal (unlock)

mutex_unlock()

4) (a) Distributed operating System

Feature	Distributed OS	Centralized OS
Control	multiple nodes	single system
resource sharing	network-based	local
fault tolerance	High	Low
Scalability	High	Limited

(b) Architecture Diagram



(c) IPC System calls

message passing:

```
Send (message);  
Receive (message);
```

Shared memory

```
shmget();  
shmat();  
shmdt();
```

5) (a) major OS function:

1) process scheduling :-

→ Allocates CPU to processes

→ Uses Algorithm

2) memory management :-

→ Allocates / Deallocates

→ Handles paging

3) Device management :-

→ control I/O Devices

→ uses drivers and buffering

(c) system call Ex:

memory allocation

```
void *ptr;
```

```
ptr = malloc(100);
```

```
free (ptr);
```

Device management:

```
int fd;
```

```
fd = open (0, RDRW);
```

```
write (fd, data, size);
```

```
close (fd);
```

(b) Block Diagram

User Application



OS Kernel



• Process scheduler

• memory manager

• Device manager

• File system



Hardware